



武汉大学测绘学院
SCHOOL OF GEODESY AND GEOMATICS

《组合导航》实验报告

实验：GNSS/INS 松组合滤波

姓名：秦旗峰

学号：2023302143029

年级：2023 级

班级：智能导航实验班 1 班

教师：朱智勤

时间：2025 年 12 月

摘要

本报告聚焦于 GNSS/INS 松组合滤波算法。本实验依托于教材附带程序和教师所给程序，分别进行 e 系和 n 系松组合代码编写或修改。通过纯惯导解算与二者对比，发现松组合性能优越，GNSS 的加入能有效抑制 IMU 误差累积，滤波的稳定性和鲁棒性也给定位带来了更大的稳定性。通过不同带估参数和机械编排方式，本报告分析了 e 系和 n 系不同松组合模式的性能差异。

关键词：松组合；导航坐标系；ECEF 坐标系

目录

1	实验目的.....	1
2	实验原理.....	2
2.1	卡尔曼滤波模型	2
2.2	GNSS/INS 松组合滤波设计	4
2.2.1	系统状态方程	4
2.2.2	GNSS 位置观测方程	6
2.2.3	GNSS 速度观测方程	7
3	实验设置.....	9
3.1	文件说明	9
3.2	参数设置	9
4	程序设计.....	10
4.1	开发环境与工具	10
4.2	核心代码	10
4.2.1	GNSS 更新函数（n 系）	10
4.2.2	误差反馈函数（n 系）	12
4.2.3	e 系松组合函数	13
5	松组合结果分析.....	15
5.1	n 系松组合	15
5.1.1	无 GNSS 测量更新	15
5.1.2	进行 GNSS 测量更新	15
5.2	e 系松组合	16
6	结论与反思.....	19

GNSS/INS 松组合滤波

1 实验目的

惯性导航是一种积分运算，其导航误差随时间积累，常需要与其他导航定位手段进行组合来解决精度发散的问题。全球导航卫星系统（GNSS）与 INS 是常用的一种组合导航系统，实现了 GNSS 与 INS 两个子系统的优势互补，以提升导航系统的精度和冗余度。GNSS/INS 组合有多种具体的实现方式，如松组合、紧组合和深组合等。GNSS/INS 松组合是在位置、速度这一信息层面的组合，组合形式简单。在工程实践中最常用扩展卡尔曼滤波来实现 GNSS 和 IMU 的数据融合解算。

本实验的主要目的为：

- (1) 熟悉 INS/GNSS 松组合滤波流程，理解时间预测和测量更新在滤波过程中分别起到的作用。
- (2) 掌握利用 MATLAB 对 INS/GNSS 松组合滤波过程进行编程解算。

2 实验原理

2.1 卡尔曼滤波模型

卡尔曼滤波算法作为一种重要的最优估计理论被广泛应用于各种领域，组合导航系统的设计是其成功应用中的一个最主要方面。卡尔曼滤波有离散型和连续型两种形式，前者可直接用于数字计算机上实现，后者更常用于卡尔曼滤波的理论性能分析。这里只介绍应用离散卡尔曼滤波算法实现 GNSS/INS 松组合解算的具体设计。

卡尔曼滤波的基本状态方程可以描述为：

$$\mathbf{X}_k = \Phi_{k,k-1}\mathbf{X}_{k-1} + \Gamma_{k-1}\mathbf{w}_{k-1} \quad (2-1)$$

式中， $\mathbf{X}_k$ 和 $\mathbf{X}_{k-1}$ 分别为 k 时刻和 $k-1$ 时刻的估计状态； $\Phi_{k,k-1}$ 为 k 时刻至 $k-1$ 时刻的状态转移矩阵； Γ_{k-1} 和 $\mathbf{W}_{k-1}$ 分别为系统噪声序列和系统噪声驱动矩阵。

若对估计状态 $\mathbf{X}_k$ 的量测满足线性关系，则对应的量测方程为：

$$\mathbf{Z}_k = \mathbf{H}_k\mathbf{X}_k + \mathbf{v}_k \quad (2-2)$$

式中， $\mathbf{H}_k$ 为量测矩阵； $\mathbf{V}_k$ 为量测噪声序列。同时系统噪声和量测噪声均服从均值白噪声的高斯分布且系统噪声和量测噪声无关：

$$\begin{aligned} E(\mathbf{w}_k) &= \mathbf{0}, Cov\{\mathbf{w}_k, \mathbf{w}_j\} = \mathbf{Q}_k\delta_{kj} \\ E(\mathbf{v}_k) &= \mathbf{0}, Cov\{\mathbf{v}_k, \mathbf{v}_j\} = \mathbf{R}_k\delta_{kj} \\ Cov\{\mathbf{w}_k, \mathbf{v}_j\} &= \mathbf{0} \end{aligned} \quad (2-3)$$

式中， $\mathbf{Q}_k$ 是 k 时刻状态过程噪声方差阵，且为非负定阵； $\mathbf{R}_k$ 为观测噪声的方差阵，且为正定阵； δ_{kj} 表示 Kronecker Delta 符号，取值规则仅由下标 k 和 j 是否相等决定：

- 当 $k = j$ 时， $\delta_{kj} = 1$ ；
- 当 $k \neq j$ 时， $\delta_{kj} = 0$ 。

标准卡尔曼滤波包括时间更新（时间预测）和量测更新两个过程。

时间更新：即上一历元的状态向量和方差协方差阵通过状态转移矩阵预测得到当前历元的状态向量和方差协方差阵。

量测更新：即利用当前历元的观测信息对预测的状态向量及其方差协方差阵进行修正。

忽略中间过程的推导，直接给出离散型卡尔曼滤波的基本公式：

(1) 状态一步预测:

$$\hat{\mathbf{X}}_{k,k-1} = \Phi_{k,k-1} \hat{\mathbf{X}}_{k-1} \quad (2-4)$$

(2) 协方差一步预测:

$$\mathbf{P}_{k,k-1} = \Phi_{k,k-1} \mathbf{P}_{k-1} \Phi_{k,k-1}^T + \Gamma_{k-1} \mathbf{Q}_k \Gamma_{k-1}^T \quad (2-5)$$

(3) 计算卡尔曼增益:

$$\mathbf{K}_k = \mathbf{P}_{k,k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k,k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (2-6)$$

(4) 状态更新:

$$\hat{\mathbf{X}}_k = \hat{\mathbf{X}}_{k,k-1} + \mathbf{K}_k (\mathbf{Z}_k - \mathbf{H}_k \hat{\mathbf{X}}_{k,k-1}) \quad (2-7)$$

(5) 协方差更新:

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k,k-1} \quad (2-8)$$

或:

$$\mathbf{P}_k^{-1} = \mathbf{P}_{k-1}^{-1} + \mathbf{H}_k^T \mathbf{R}_k^{-1} \mathbf{H}_k \quad (2-9)$$

为减小计算舍入误差对滤波的影响, 保证对称正定性, 一般改写如下:

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k,k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^T \quad (2-10)$$

相比于式(2-8)(2-27)和式(2-9)(2-28), 公式(2-10)错误!未找到引用源。在进行协方差更新时求逆操作较少并且可以保证矩阵的正定性, 因此被广泛使用。

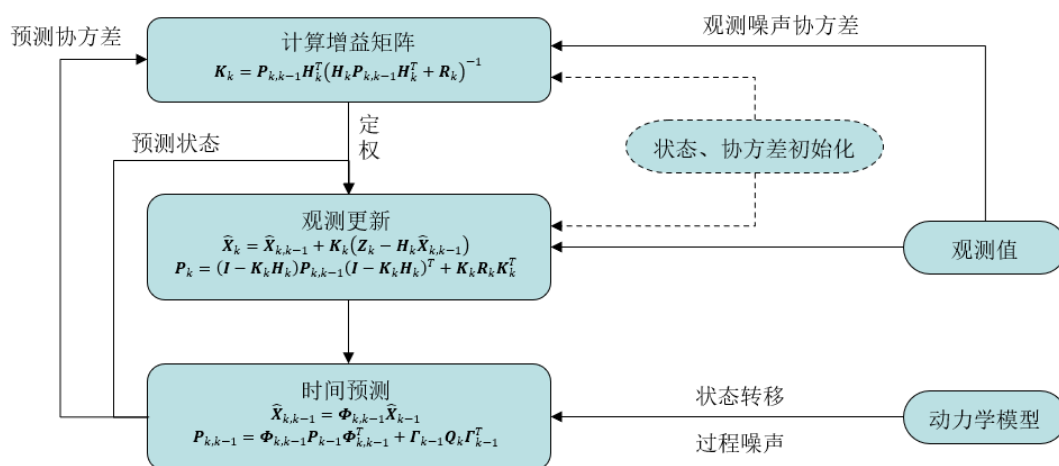


图 2-1 卡尔曼滤波流程

以上公式即为离散卡尔曼滤波的基本方程。只需要给定状态初始值 $\hat{\mathbf{X}}_0$ 和对应的初始协方差 $\mathbf{P}_0$, 设计每个时刻的转移矩阵和过程噪声矩阵, 根据对应 k 时刻的量测 $\mathbf{Z}_k$, 就能够计算出 k 时刻的状态估计 $\hat{\mathbf{X}}_k$ 。

2.2 GNSS/INS 松组合滤波设计

GNSS/INS 松组合卡尔曼滤波算法设计的关键在于：建立离散时间系统误差状态模型，然后构建误差状态的线性观测方程。有了系统模型和观测方程则可直接使用卡尔曼滤波的基本方程进行组合导航解算，即完成了组合导航的算法设计。

2.2.1 系统状态方程

GNSS/INS 松组合常采用误差状态卡尔曼滤波（间接卡尔曼滤波）进行组合导航解算，以解决系统的非线性问题。因此，卡尔曼滤波的状态向量包含导航状态误差和传感器误差，定义为：

$$\delta \mathbf{x}(t) = [(\delta \mathbf{r}^n)^T \quad (\delta \mathbf{v}^n)^T \quad \boldsymbol{\phi}^T \quad \mathbf{b}_g^T \quad \mathbf{b}_a^T \quad \mathbf{s}_g^T \quad \mathbf{s}_a^T]^T \quad (2-11)$$

式中， $\delta \mathbf{r}^n$ 为惯导位置误差向量； $\delta \mathbf{v}^n$ 为惯导速度误差向量； $\boldsymbol{\phi}$ 为姿态误差向量； $\mathbf{b}_g$ 为三轴陀螺零偏向量； $\mathbf{b}_a$ 为三轴加速度计零偏向量； $\mathbf{s}_g$ 为陀螺比例因子误差向量； $\mathbf{s}_a$ 为加速度计比例因子误差向量。向量 $\delta \mathbf{x}$ 中的各分量均为时间的函数。式(2-11)表示将陀螺和加速度计的零偏及比例因子误差增广到卡尔曼滤波的状态量中进行估计。

为得到系统状态方程，首先需要得到 $\delta \mathbf{x}(t)$ 的连续时间微分方程，写作如下形式：

$$\delta \dot{\mathbf{x}}(t) = \mathbf{F}(t)\delta \mathbf{x}(t) + \mathbf{G}(t)\mathbf{w}(t) \quad (2-12)$$

对 $\delta \mathbf{x}(t)$ 向量求导即对向量的各分量分别对时间 t 求导，其中位置、速度和姿态误差微分方程如下：

$$\delta \dot{\mathbf{r}}^n = -\boldsymbol{\omega}_{en}^n \times \delta \mathbf{r}^n + \delta \boldsymbol{\theta} \times \mathbf{v}^n + \delta \mathbf{v}^n \quad (2-13)$$

$$\delta \dot{\mathbf{v}}^n = \mathbf{C}_b^n \delta \mathbf{f}^n + \mathbf{f}^n \times \boldsymbol{\phi} - (2\boldsymbol{\omega}_{ie}^n + \boldsymbol{\omega}_{en}^n) \times \delta \mathbf{v}^n + \mathbf{v}^n \times (2\delta \boldsymbol{\omega}_{ie}^n + \delta \boldsymbol{\omega}_{en}^n) + \delta \mathbf{g}_p^n \quad (2-14)$$

$$\dot{\boldsymbol{\phi}} = -\boldsymbol{\omega}_{in}^n \times \boldsymbol{\phi} + \delta \boldsymbol{\omega}_{in}^n - \delta \boldsymbol{\omega}_{ib}^n \quad (2-15)$$

陀螺和加速度计零偏及比例因子误差均建模为一阶高斯马尔可夫过程：

$$\begin{cases} \dot{\mathbf{b}}_g(t) = -\frac{1}{T_{gb}} \mathbf{b}_g(t) + \mathbf{w}_{gb}(t) \\ \dot{\mathbf{b}}_a(t) = -\frac{1}{T_{ab}} \mathbf{b}_a(t) + \mathbf{w}_{ab}(t) \\ \dot{\mathbf{s}}_g(t) = -\frac{1}{T_{gs}} \mathbf{s}_g(t) + \mathbf{w}_{gs}(t) \\ \dot{\mathbf{s}}_a(t) = -\frac{1}{T_{as}} \mathbf{s}_a(t) + \mathbf{w}_{as}(t) \end{cases} \quad (2-16)$$

式中, T_{gb} 、 T_{ab} 、 T_{gs} 和 T_{as} 为一阶高斯马尔可夫过程的相关时间; $\mathbf{w}_{gb}(t)$ 、 $\mathbf{w}_{ab}(t)$ 、 $\mathbf{w}_{gs}(t)$ 和 $\mathbf{w}_{as}(t)$ 为一阶高斯马尔可夫过程的驱动白噪声。

根据 $\delta\dot{\mathbf{x}}(t)$ 的表达式, 容易写出连续时间系统噪声向量 $\mathbf{w}(t)$ 和矩阵 $\mathbf{G}(t)$ 。

$$\mathbf{w} = [\mathbf{w}_v^T \quad \mathbf{w}_\phi^T \quad \mathbf{w}_{gb}^T \quad \mathbf{w}_{ab}^T \quad \mathbf{w}_{gs}^T \quad \mathbf{w}_{as}^T]^T \quad (2-17)$$

$$E[\mathbf{w}(t)\mathbf{w}^T(\tau)] = \mathbf{q}(t)\delta(t - \tau) \quad (2-18)$$

其中 $\mathbf{w}_v$ 和 $\mathbf{w}_\phi$ 分别为加速度计和陀螺的测量白噪声, $\mathbf{w}_{gb}(t)$ 、 $\mathbf{w}_{ab}(t)$ 、 $\mathbf{w}_{gs}(t)$ 和 $\mathbf{w}_{as}(t)$ 为陀螺和加速度计零偏及比例因子误差建模的驱动白噪声。为方便使用离散时间卡尔曼滤波的基本方程, 需要对式(2-12)进行离散化处理, 离散化后的系统状态方程为:

$$\delta\mathbf{x}_k = \Phi_{k/k-1}\delta\mathbf{x}_{k-1} + \mathbf{w}_{k-1} \quad (2-19)$$

记离散化时间间隔 $\Delta t = t_k - t_{k-1}$, 当 $\mathbf{F}(t)$ 在较短的积分区间 $[t_{k-1}, t_k]$ 内变化不太剧烈时, 且设 $\mathbf{F}(t_{k-1})\Delta t \ll \mathbf{I}$, 则 $\Phi_{k/k-1}$ 可近似为:

$$\Phi_{k/k-1} = \exp\{\mathbf{F}(t_{k-1})\Delta t\} \approx \mathbf{I} + \mathbf{F}(t_{k-1})\Delta t \quad (2-20)$$

$$\mathbf{w}_{k-1} = \int_{t_{k-1}}^{t_k} \Phi_{k/t} \mathbf{G}(t) \mathbf{w}(t) dt \quad (2-21)$$

$\mathbf{w}_{k-1}$ 的二阶矩 $\mathbf{Q}_k = E[\mathbf{w}_{k-1}\mathbf{w}_{k-1}^T]$, 如下计算:

$$\begin{aligned} \mathbf{Q}_k &= E \left\{ \int_{t_{k-1}}^{t_k} \Phi_{k/t} \mathbf{G}(t) \mathbf{w}(t) dt \left[\int_{t_{k-1}}^{t_k} \Phi_{k/\tau} \mathbf{G}(\tau) \mathbf{w}(\tau) d\tau \right]^T \right\} \\ &= \int_{t_{k-1}}^{t_k} \Phi_{k/t} \mathbf{G}(t) \mathbf{q}(t) \mathbf{G}^T(t) \Phi_{k/t}^T dt \end{aligned} \quad (2-22)$$

同样, 当 $\mathbf{G}(t)$ 在较短的积分区间 $[t_{k-1}, t_k]$ 内变化不太剧烈时, $\mathbf{Q}_k$ 可简化为梯形积分:

$$\mathbf{Q}_k \approx \frac{1}{2} [\Phi_{k/k-1} \mathbf{G}(t_{k-1}) \mathbf{q}(t_{k-1}) \mathbf{G}^T(t_{k-1}) \Phi_{k/k-1}^T + \mathbf{G}(t_k) \mathbf{q}(t_k) \mathbf{G}^T(t_k)] \Delta t \quad (2-23)$$

从上述推导可知 $\mathbf{w}_k$ 为白噪声序列, (2-19)描述的等效离散系统满足离散型卡尔曼滤波的基本方程要求。

2.2.2 GNSS 位置观测方程

GNSS 定位解算给出的是天线相位中心（或其它参考点）的位置坐标，INS 机械编排给出的是 IMU 测量中心的导航结果，二者在物理上不重合（如图 1 所示）。

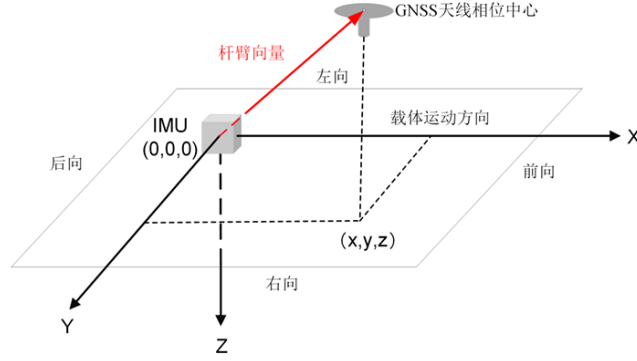


图 2-2 GNSS 杆臂示意图

因此在数据融合解算时需进行杆臂效应改正。GNSS 天线相位中心与 IMU 测量中心之间的位置转换关系式为：

$$\mathbf{r}_G^n = \mathbf{r}_I^n + \mathbf{D}_R^{-1} \mathbf{C}_b^n \mathbf{l}^b \quad (2-24)$$

$$\mathbf{D}_R^{-1} = \text{diag} \left(\left[\frac{1}{R_M + h} \quad \frac{1}{(R_N + h) \cos \varphi} \quad -1 \right]^T \right) \quad (2-25)$$

式(2-24)中 $\mathbf{r}_G^n$ 表示 GNSS 天线相位中心的实际位置， $\mathbf{r}_I^n$ 表示 IMU 测量中心的实际位置，均用纬度 φ 、经度 λ 和高程 h 的大地坐标表示。 $\mathbf{l}^b$ 为 GNSS 天线的杆臂向量，即 IMU 测量中心指向 GNSS 天线相位中心的向量在 b 系下的投影。 $\mathbf{l}^b$ 可以通过精密测量手段进行测量标定，一般认为其误差可以忽略。

在实际导航应用中，考虑各导航状态的误差，式(2-24)可以写作：

$$\hat{\mathbf{r}}_G^n = \hat{\mathbf{r}}_I^n + \mathbf{D}_R^{-1} \hat{\mathbf{C}}_b^n \mathbf{l}^b \quad (2-26)$$

式中带上标 $\hat{\cdot}$ 的变量表示带有误差的计算值。 $\hat{\mathbf{r}}_I^n$ 为惯导解算的 IMU 中心位置向量， $\hat{\mathbf{r}}_G^n$ 为惯导推算的 GNSS 天线相位中心的位置向量， $\hat{\mathbf{C}}_b^n$ 为惯导解算的姿态矩阵。在上式扰动分析时忽略了位置误差对矩阵 $\mathbf{D}_R^{-1}$ 的影响，因为其影响非常小。回顾误差扰动分析，将姿态误差的定义式重写如下：

$$\hat{\mathbf{C}}_b^n = [\mathbf{I} - (\boldsymbol{\phi} \times)] \mathbf{C}_b^n \quad (2-27)$$

$$\hat{\mathbf{r}}_I^n = \mathbf{r}_I^n + \mathbf{D}_R^{-1} \delta \mathbf{r}^n \quad (2-28)$$

将式(2-24)、(2-27)和(2-28)代入式(2-26)，整理可得：

$$\begin{aligned}\hat{\mathbf{r}}_G^n &= \mathbf{r}_I^n + \mathbf{D}_R^{-1} \delta \mathbf{r}^n + \mathbf{D}_R^{-1} [\mathbf{I} - (\boldsymbol{\phi} \times)] \mathbf{C}_b^n \mathbf{l}^b \\ &= \mathbf{r}_G^n + \mathbf{D}_R^{-1} \delta \mathbf{r}^n + \mathbf{D}_R^{-1} [(\mathbf{C}_b^n \mathbf{l}^b) \times \boldsymbol{\phi}]\end{aligned}\quad (2-29)$$

GNSS 定位解算得到的 GNSS 天线相位中心的位置表示为:

$$\tilde{\mathbf{r}}_G^n = \mathbf{r}_G^n - \mathbf{D}_R^{-1} \mathbf{n}_r \quad (2-30)$$

式中, 上标~表示为观测值, $\tilde{\mathbf{r}}_G^n$ 为 GNSS 位置观测量, $\mathbf{n}_r$ 为 GNSS 的位置误差。一般地, 为简化处理常将 GNSS 位置测量值的误差建模为高斯白噪声序列, 即 $\mathbf{n}_{r,k} \sim N(0, \mathbf{R}_k)$, $\mathbf{R}_k = E[\mathbf{n}_{r,k} \mathbf{n}_{r,k}^T]$ 。

观测误差的方差 $\mathbf{R}_k$ 一般可从 GNSS 定位解算程序/软件中得到。因此, 对应的观测向量表示为 INS 推算的位置与 GNSS 位置观测之差, 易得:

$$\delta \mathbf{z}_r = \mathbf{D}_R (\hat{\mathbf{r}}_G^n - \tilde{\mathbf{r}}_G^n) \approx \delta \mathbf{r}^n + (\mathbf{C}_b^n \mathbf{l}^b) \times \boldsymbol{\phi} + \mathbf{n}_r \quad (2-31)$$

因此, GNSS 位置观测方程可写作如下形式:

$$\delta \mathbf{z}_r = \mathbf{H}_r \delta \mathbf{x} + \mathbf{n}_r \quad (2-32)$$

式中:

$$\mathbf{H}_r = [\mathbf{I}_3 \quad \mathbf{0}_3 \quad (\mathbf{C}_b^n \mathbf{l}^b) \times \quad \mathbf{0}_3 \quad \mathbf{0}_3 \quad \mathbf{0}_3 \quad \mathbf{0}_3] \quad (2-33)$$

实际解算时, 观测矩阵 $\mathbf{H}_r$ 中的 $\mathbf{C}_b^n$ 用最新时刻的惯导姿态 $\hat{\mathbf{C}}_b^n$ 进行赋值。

2.2.3 GNSS 速度观测方程

GNSS 利用多普勒观测值可以解算出天线相位中心处的三维速度观测值 $\mathbf{v}_G^n$ 。IMU 中心与 GNSS 天线相位中心之间的速度转换关系为:

$$\mathbf{v}_G^n = \mathbf{v}_I^n - (\boldsymbol{\omega}_{in}^n \times) \mathbf{C}_b^n \mathbf{l}^b - \mathbf{C}_b^n (\mathbf{l}^b \times \boldsymbol{\omega}_{ib}^b) \quad (2-34)$$

式中, $\mathbf{v}_I^n$ 和 $\mathbf{v}_G^n$ 分别表示 IMU 中心和 GNSS 天线相位中心的实际速度向量, $\boldsymbol{\omega}_{in}^n$ 表示 n 系相对于 i 系的转动角速度向量在 n 系下的投影, $(\boldsymbol{\omega}_{in}^n \times)$ 表示向量 $\boldsymbol{\omega}_{in}^n$ 对应的反对称矩阵; $\boldsymbol{\omega}_{ib}^b$ 为陀螺理论输出角速度向量。对于普通运动场景, 只要速度和杆臂不是特别大, 等式右边第二项 $(\boldsymbol{\omega}_{in}^n \times) \mathbf{C}_b^n \mathbf{l}^b$ 的幅值很小, 很多情况下都可直接忽略。在实际导航应用中, 式(2-34)可写作:

$$\hat{\mathbf{v}}_G^n = \hat{\mathbf{v}}_I^n - (\boldsymbol{\omega}_{in}^n \times) \hat{\mathbf{C}}_b^n \mathbf{l}^b - \hat{\mathbf{C}}_b^n (\mathbf{l}^b \times \hat{\boldsymbol{\omega}}_{ib}^b) \quad (2-35)$$

回顾惯导速度误差和陀螺角速度测量误差的定义:

$$\hat{\mathbf{v}}_I^n = \mathbf{v}_I^n + \delta \mathbf{v}^n \quad (2-36)$$

$$\hat{\boldsymbol{\omega}}_{ib}^b = \boldsymbol{\omega}_{ib}^b + \delta \boldsymbol{\omega}_{ib}^b \quad (2-37)$$

$$\delta \omega_{ib}^b = \mathbf{b}_g + \text{diag}(\omega_{ib}^b) \mathbf{s}_g + \mathbf{w}_\phi \quad (2-38)$$

式中, $\mathbf{b}_g$ 为陀螺零偏, $\mathbf{s}_g = [s_{g,x} \ s_{g,y} \ s_{g,z}]^T$ 为陀螺比例因子误差向量, $\mathbf{w}_\phi$ 为陀螺测量白噪声。将式(2-27)、(2-36)和(2-37)代入式(2-35), 省略关于误差的二阶项, 整理可得:

$$\begin{aligned} \hat{\mathbf{v}}_G^n &= \mathbf{v}_I^n + \delta \mathbf{v}^n - (\omega_{in}^n \times) [I - (\boldsymbol{\phi} \times)] \mathbf{C}_b^n \mathbf{l}^b - [I - (\boldsymbol{\phi} \times)] \mathbf{C}_b^n [\mathbf{l}^b \times (\omega_{ib}^b + \delta \omega_{ib}^b)] \\ &\approx \mathbf{v}_G^n + \delta \mathbf{v}^n - (\omega_{in}^n \times) (\mathbf{C}_b^n \mathbf{l}^b \times) \boldsymbol{\phi} - [\mathbf{C}_b^n (\mathbf{l}^b \times \omega_{ib}^b)] \times \boldsymbol{\phi} - \mathbf{C}_b^n (\mathbf{l}^b \times) \delta \omega_{ib}^b \end{aligned} \quad (2-39)$$

GNSS 解算的天线相位中心速度 (GNSS 速度观测值) 为:

$$\tilde{\mathbf{v}}_G^n = \mathbf{v}_G^n - \mathbf{n}_v \quad (2-40)$$

速度观测向量可表示为 INS 推算的速度与 GNSS 解算的速度之差:

$$\begin{aligned} \delta \mathbf{z}_v &= \hat{\mathbf{v}}_G^n - \tilde{\mathbf{v}}_G^n \\ &= \delta \mathbf{v}^n - (\omega_{in}^n \times) (\mathbf{C}_b^n \mathbf{l}^b \times) \boldsymbol{\phi} - [\mathbf{C}_b^n (\mathbf{l}^b \times \omega_{ib}^b)] \times \boldsymbol{\phi} - \mathbf{C}_b^n (\mathbf{l}^b \times) \delta \omega_{ib}^b + \mathbf{n}_v \end{aligned} \quad (2-41)$$

因此, GNSS 速度观测方程可写作如下形式:

$$\delta \mathbf{z}_v = \mathbf{H}_v \delta \mathbf{x} + \mathbf{n}_v \quad (2-42)$$

将式(2-38)代入式(2-41), 容易写出相应的观测矩阵 $\mathbf{H}_v$:

$$\mathbf{H}_v = [\mathbf{0}_3 \quad \mathbf{I}_3 \quad \mathbf{H}_{v3} \quad -(\mathbf{C}_b^n \mathbf{l}^b \times) \quad \mathbf{0}_3 \quad \mathbf{H}_{v6} \quad \mathbf{0}_3] \quad (2-43)$$

式中:

$$\begin{cases} \mathbf{H}_{v3} = -(\omega_{in}^n \times) (\mathbf{C}_b^n \mathbf{l}^b \times) - [\mathbf{C}_b^n (\mathbf{l}^b \times \omega_{ib}^b)] \times \\ \mathbf{H}_{v6} = -\mathbf{C}_b^n (\mathbf{l}^b \times) \text{diag}(\omega_{ib}^b) \end{cases} \quad (2-44)$$

3 实验设置

3.1 文件说明

(1) IMU 数据格式: (数据文件为 IMU_incre.bin)

IMU_incre.bin 是 7 列二进制文件, double 类型; 每一行数据为: 时间(GPS 周内秒), 三轴陀螺增量(rad), 三轴加表增量(m/s); IMU 采样频率为 50Hz。

(2) GNSS 数据格式: (数据文件为 gnss_RTD_vel.txt)

gnss_RTD_vel.txt 是 13 列文本文件, 数据分别为: 时间(GPS 周内秒), 经纬高(deg, deg, m), 北东地速度(m/s), 北东地位置 std(m), 北东地速度 std(m/s)。

3.2 参数设置

实验卡尔曼滤波参数包括 GNSS 和 IMU 的噪声设置, 用于卡尔曼滤波求解。

实验误差参数设置如下 (相关时间1h):

表 3-1 实验参数设置

参数	含义	取值
initposstd	初始位置误差	[0.5 0.5 1.0]m
initvelstd	初始速度误差	[0.1 0.1 0.1]m/s
initattstd	初始姿态误差	[0.5 0.5 1.0]deg
initgyrbiasstd	初始陀螺零偏误差	[50 50 50]deg/h
initaccbiasstd	初始加速度计零偏误差	[250 250 250]mGal
initgyrscalestd	初始陀螺比例因子误差	[1000 1000 1000]ppm
initaccscalestd	初始加速度计比例因子误差	[1000 1000 1000]ppm
gyrarw	角度随机游走	0.24deg/s/sqrt(h)
accvrw	速度随机游走	0.24m/s/sqrt(h)
gyrbiasstd	陀螺零偏不稳定性	50deg/h
accbiasstd	加速度计零偏不稳定性	250mGal
gyrscalestd	陀螺比例因子不稳定性	1000ppm
accscalestd	加速度计比例因子不稳定性	1000ppm

4 程序设计

4.1 开发环境与工具

- (1) 开发语言: matlab
- (2) 开发工具: MATLAB R2024b

4.2 核心代码

4.2.1 GNSS 更新函数 (n 系)

`function kf = GNSSUpdate(navstate, gnssdata, kf, antlever, usegnssvel, thisimu, dt)`

此函数为 GNSS 观测更新函数，传入导航状态 `navstate`，GNSS 观测值 `gnssdata`，卡尔曼滤波器 `kf`，杆臂数据 `antlever`，是否使用 GNSS 测量的速度进行更新 `usegnssvel`，当前历元 IMU 数据 `thisimu` 和间隔时间 `dt`；解算出 GNSS 更新后的滤波器状态 `kf`。

在函数内根据 GNSS 数据定位和定速误差判断是否进行 GNSS 测量更新，若进行更新则依照松组合原理进行观测方程构建，滤波求解。

```
function kf = GNSSUpdate(navstate, gnssdata, kf, antlever, usegnssvel, thisimu, dt)
    param = Param();
    %% GNSS position update
    % abandon gnss vel outlier
    gnssposstd = gnssdata(8:10, 1);
    if gnssposstd(1, 1) > 5 || gnssposstd(2, 1) > 5 || gnssposstd(3, 1) > 10
        disp(['WARNING: Abandon gnss position measurement at: ', num2str(gnssdata(1, 1))]);
    else
        % measurement innovation
        % DR:BLH 转 NED 矩阵
        DR = diag([navstate.Rm + navstate.pos(3), (navstate.Rn + navstate.pos(3))*cos(navstate.pos(1)), -1]);
        Z = DR*(navstate.pos - gnssdata(2:4, 1))+navstate.Cbn*antlever;%N 系下的 NED
        % measurement matrix and noise matrix
        R = diag(power(gnssdata(8:10, 1), 2));%m m m
        H = zeros(3, kf.RANK);
```

```

H(1:3, 1:3) = eye(3);
H(1:3, 7:9) = skew(navstate.Cbn * antlever);

% update covariance and state vector
K = kf.P * H' / (H * kf.P * H' + R);
kf.x = kf.x + K*(Z - H*kf.x);
kf.P=(eye(kf.RANK) - K*H) * kf.P * (eye(kf.RANK) - K*H)' + K * R * K';
end
%% GNSS velocity update
if usegnssvel
    omega_Earth=7.292115E-5;%rad
    % abandon gnss vel outlier
    gnssvelstd = gnssdata(11:13, 1);
    if gnssvelstd(1, 1) > 0.5 || gnssvelstd(2, 1) > 0.5 || gnssvelstd(3,
1) > 0.5
        disp(['WARNING: Abandon gnss velocity measurement at: ', num2str(
gnssdata(1, 1))]);
    else
        %measurement innovation, noise, matrix
        gnss_vel_ned=gnssdata(5:7,1);
        gyro_b=thisimu(2:4,1)/dt;
        omega_in_n(1,1)=omega_Earth*cos(navstate.pos(1))+navstate.vel(2)/
(navstate.Rn+navstate.pos(3));
        omega_in_n(2,1)=-navstate.vel(1)/(navstate.Rm+navstate.pos(3));
        omega_in_n(3,1)=-omega_Earth*sin(navstate.pos(1))-
navstate.vel(2)*tan(navstate.pos(1))/(navstate.Rn+navstate.pos(3));
        antlever_vel_ned=navstate.Cbn*skew(gyro_b)*antlever-
skew(omega_in_n)*navstate.Cbn*antlever;
        ins_ant_vel_ned=navstate.vel+antlever_vel_ned;
        Z_vel=ins_ant_vel_ned-gnss_vel_ned;
        H_v3=zeros(3,3);
        H_v6=zeros(3,3);
        H_v3(1:3,1:3)=-skew(omega_in_n)*skew(navstate.Cbn*antlever)-
skew(navstate.Cbn*(skew(antlever)*gyro_b));
        H_v6(1:3,1:3)=-navstate.Cbn*skew(antlever)*diag(gyro_b);
        R_vel=diag(power(gnssdata(11:13,1),2));
        H_vel=zeros(3,kf.RANK);
        H_vel(1:3,4:6)=eye(3);
        H_vel(1:3,7:9)=H_v3(1:3,1:3);
        H_vel(1:3,10:12)=-skew(navstate.Cbn*antlever);
        H_vel(1:3,16:18)=H_v6(1:3,1:3);
        K_vel=kf.P*H_vel'/(H_vel*kf.P*H_vel'+R_vel);

```

```

        kf.x=kf.x+K_vel*(Z_vel-H_vel*kf.x);
        kf.P=(eye(kf.RANK)-K_vel*H_vel)*kf.P*(eye(kf.RANK)-
K_vel*H_vel)'+K_vel*R_vel*K_vel';
    end
end
end

```

4.2.2 误差反馈函数（n 系）

```
function [kf, navstate] = ErrorFeedback(kf, navstate)
```

此函数为误差反馈函数，功能为松组合闭环校正，利用计算得到的误差信息校正到 IMU 推算结果上。传入滤波器 `kf` 和导航状态 `navstate`，在函数内对其进行更新。

```

function [kf, navstate] = ErrorFeedback(kf, navstate)
    % position and velocity
    DR = diag([navstate.Rm + navstate.pos(3), (navstate.Rn + navstate.pos(3))
*cos(navstate.pos(1)), -1]);
    DR_inv = inv(DR);
    navstate.pos = navstate.pos - DR_inv * kf.x(1:3, 1);
    navstate.vel = navstate.vel - kf.x(4:6, 1);
    % attitude
    qpn = Rotvec2Quat(kf.x(7:9, 1));
    navstate.qbn = QuaternionMultiply(qpn, navstate.qbn);
    navstate.Cbn = Quaternion2DCM(navstate.qbn);
    navstate.qtt = DCM2Euler(navstate.Cbn);
    %   navstate.Cbn = (eye(3) + skew(kf.x(7:9, 1))) * navstate.Cbn;
    %   navstate.att = DCM2Euler(navstate.Cbn);
    %   navstate.qbn = Euler2Quaternion(navstate.att);
    % imu error
    navstate.gyrbias = navstate.gyrbias + kf.x(10:12, 1);
    navstate.accbias = navstate.accbias + kf.x(13:15, 1);
    navstate.gyrscale = navstate.gyrscale + kf.x(16:18, 1);
    navstate.accscale = navstate.accscale + kf.x(19:21, 1);
    % update some parameters
    navstate.Rm = getRm(navstate.pos(1));
    navstate.Rm = getRm(navstate.pos(1));
    navstate.gravity = getGravity(navstate.pos(1), navstate.pos(3));
    % reset state vector
    kf.x = zeros(kf.RANK, 1);
end

```


4.2.3 e 系松组合函数

e 系松组合函数由教材附带函数改编而来，以适应数据接口。

```
function [final_navstate, final_kf] = Loosely_coupled_INS_GNSS_qqf(imudata, g
nssdata, kf, navstate, cfg)
```

此函数输入惯导数据 `imudata` 和 GNSS 数据 `gnssdata`，卡尔曼滤波器 `kf`，导航状态 `navstate` 和配置参数 `cfg`。在此函数内部进行 ECEF 惯导编排解算，GNSS 观测值更新等函数调用，实现 e 系松组合。

主要实现代码如下：

```
function [final_navstate, final_kf] = Loosely_coupled_INS_GNSS_qqf(imudata, g
nssdata, kf, navstate, cfg)
param = Param();
gnss_index=1;
last_gnss_time=gnssdata(gnss_index,1);
for imu_index = 2:size(imudata, 1)
    % IMU 机械编排
    curr_imu = imudata(imu_index, :);
    last_imu = imudata(imu_index-1, :);
    dt_imu = curr_imu(1) - last_imu(1);
    dt_gnss=gnssdata(2,1)-gnssdata(1,1);
    omega_ib_b = (curr_imu(2:4) / dt_imu - navstate.gyrbias) ./ (ones(3,1) );
    f_ib_b      = (curr_imu(5:7) / dt_imu - navstate.accbias) ./ (ones(3,1) );
    [navstate.pos, navstate.vel, navstate.Cbe] = Nav_equations_ECEF(dt_imu, n
avstate.pos, navstate.vel, navstate.Cbe, f_ib_b, omega_ib_b);
    if gnss_index <= size(gnssdata, 1) && abs(curr_imu(1) - last_gnss_time-
1.0) <1e-6
        % GNSS 测量更新
        gnss_index = gnss_index + 1;
        last_gnss_time=gnssdata(gnss_index,1);
        fprintf('在 t=%.3f 执行 GNSS 更新\n', curr_imu(1));
        gnss_pos = gnssdata(gnss_index, 2:4)';
        gnss_vel = gnssdata(gnss_index, 5:7)';
        pos_sd_n = diag([gnssdata(gnss_index,8)^2, gnssdata(gnss_index,9)^2,
gnssdata(gnss_index,10)^2]);
        vel_sd_n = diag([gnssdata(gnss_index,11)^2, gnssdata(gnss_index,12)^2
, gnssdata(gnss_index,13)^2]);
        [lat,lon,~,~,~] = ECEF_to_NED(navstate.pos,navstate.vel,navstate.Cbe);
        sin_lat = sin(lat); cos_lat = cos(lat);
        sin_lon = sin(lon); cos_lon = cos(lon);
```

```

C_e_n = [-sin_lat*cos_lon, -sin_lat*sin_lon, cos_lat;
         -sin_lon,          cos_lon,          0;
         -cos_lat*cos_lon, -cos_lat*sin_lon, -sin_lat];
LC_KF_config.pos_meas_SD = sqrt(diag(C_e_n * pos_sd_n * C_e_n'));
LC_KF_config.vel_meas_SD = sqrt(diag(C_e_n * vel_sd_n * C_e_n'));
[navstate.Cbe,navstate.vel,navstate.pos,est_IMU_bias,...
kf.P] = LC_KF_Epoch(gnss_pos,gnss_vel,dt_gnss,...
navstate.Cbe,navstate.vel,navstate.pos,[navstate.accbias; navstate.gy
rbias],kf.P,f_ib_b,lat,LC_KF_config);
navstate.accbias = est_IMU_bias(1:3);
navstate.gyrbias = est_IMU_bias(4:6);
end
end
end

```

其 ECEF 编排函数和 GNSS 更新函数直接调用教材附带程序函数。

5 松组合结果分析

5.1 n 系松组合

n 系松组合编排是在 NED 坐标系下进行组合导航松组合定位解算。解算结果保存在 NavResult_GNSSVEL.nav，与参考真值 ref_out.nav 对比。

5.1.1 无 GNSS 测量更新

n 系下只进行 IMU 机械编排进行解算，与参考真值进行对比。

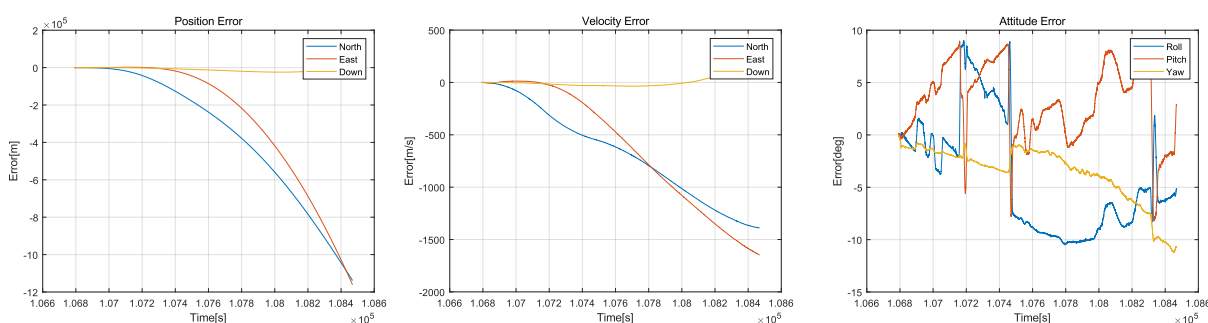


图 5-1 定位误差(无 GNSS 测量更新)

在未进行 GNSS 测量更新时，IMU 机械编排解算误差呈现发散情况。在短时间内位置误差可以漂移到 10^5m 数量级，水平速度误差在数分钟内便达到千米每秒水平。图 5-1 说明了在无任何外部数据进行修正时纯惯导解算误差快速累积，短时间内定位结果便失去其可靠性。

5.1.2 进行 GNSS 测量更新

松组合是直接利用 GNSS 定位结果作为修正信息对 IMU 解算结果加以修正。其定位结果与真值对比误差如下图：

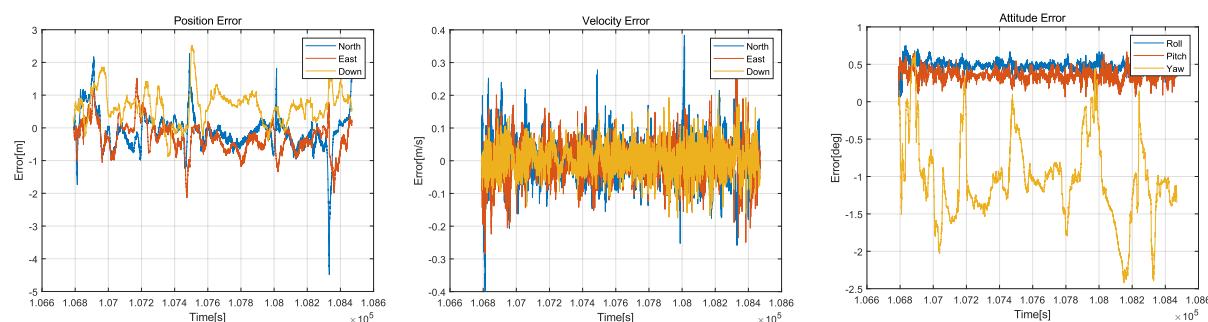


图 5-2 定位误差（GNSS 松组合）

通过图 5-1 和图 5-2 对比发现，当加入 GNSS 测量更新后定位、定速与定姿误差显著下降，三维定位误差控制在 $5m$ 以内，三维速度误差小于 $0.5m/s$ ，姿态误差以及姿态波动情况也相较于未加入 GNSS 时同样明显减少。

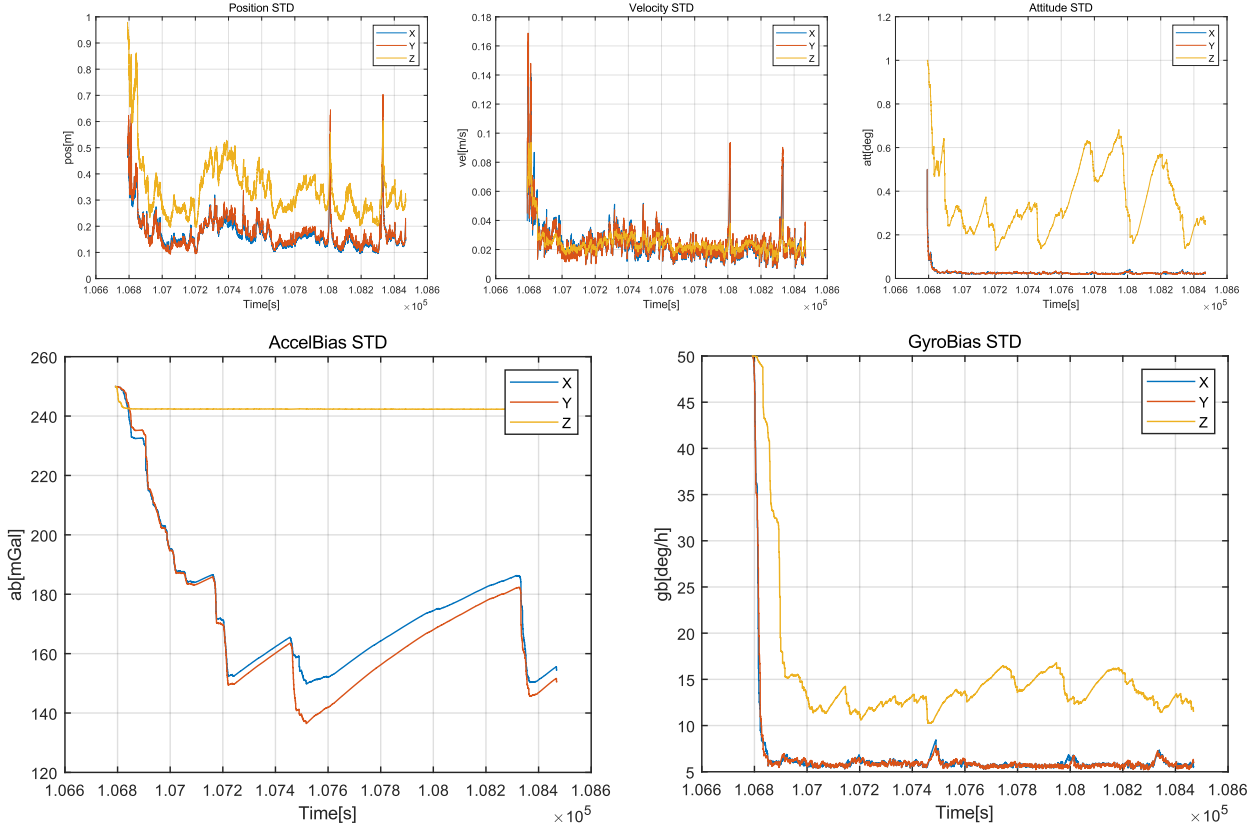


图 5-3 n 系松组合状态参数标准差

n 系松组合状态参数图 5-3 展示的是各状态参数方差（不确定度）的时序图，随着滤波时间的增加，各状态方差总体上呈现衰减趋势并趋于稳定，当由于 GNSS 观测值不佳而放弃 GNSS 更新时，方差会相应上升，足以体现 GNSS 测量更新带来的定位准确度提高。上图俯仰角和横滚角误差及波动情况减少更为明显，航向角由于 IMU 与地面相对情况不断变化，受到噪声影响更大，波动与误差相比于水平姿态角更大。

加入了 GNSS 观测值进行观测更新后，IMU 的累积误差得到了及时修正，利用 GNSS 精度较高的观测值对 IMU 机械编排不断修正，提供误差信息，维持误差处于较低水平。

5.2 e 系松组合

e 系松组合，即 GNSS 和 IMU 均在 ECEF 系下进行解算和机械编排的组合定位。

相比于 n 系松组合， e 系下进行组合导航观测模型更为简洁，定位无纬度差异，不会存在高纬度定位奇点。

定位结果与参考真值相比，解算误差如下图所示：

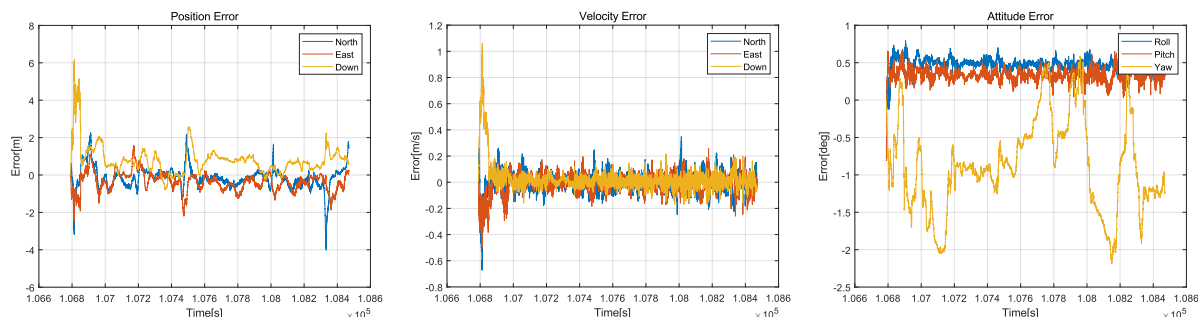


图 5-4 ECEF 系松组合定位误差

本实验 e 系松组合修改的是教材附带程序，与 5.1 节 n 系的松组合相比，没有估计加速度计和陀螺的比例因子误差，进行机械编排时也利用的是单历元 IMU 数据的简化更新。相比于图 5-2 所示解算误差， e 系下的波动情况较为明显，但总体上定位误差较为相似。

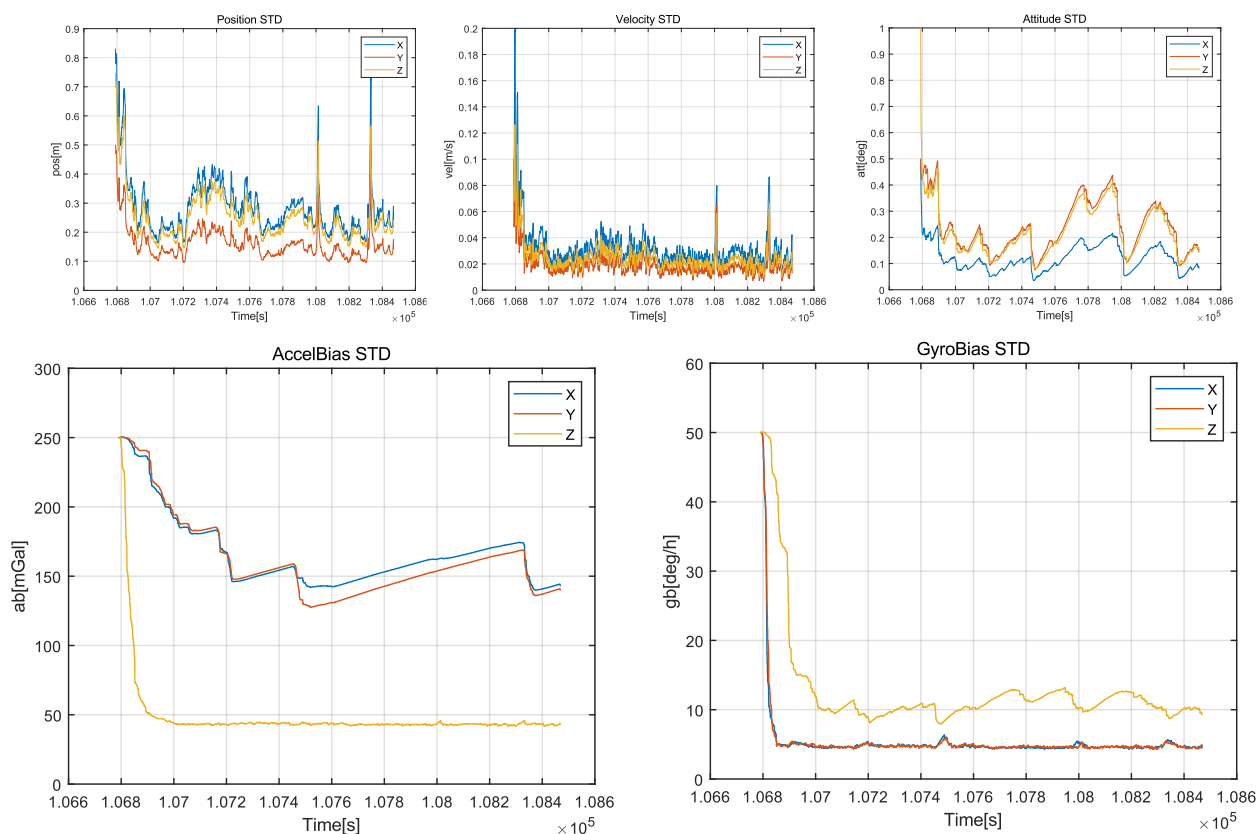


图 5-5 ECEF 松组合状态参数标准差

图 5-5 是 e 系松组合状态参数标准差，由于其估计参数较少且 IMU 机械编排算法

较为简便，相较于 n 系结果，各参数波动更为剧烈，但总体上滤波结果较好，标准差不断减少并收敛。

6 结论与反思

本文对在 n 系和 e 系下松组合定位效果展开了分析。一方面与不加入 GNSS 观测信息进行对比，另一方面在 n 系和 e 系下分别进行松组合定位并进行分析对比。

本实验 n 系和 e 系松组合采用了闭环校正，在一个函数内完成误差估计和状态校正，而误差状态不会传递下去，而是直接用于校正待估参数，具有较高的数值稳定性。

当未使用 GNSS 观测值进行纯惯导解算时定位误差快速发散，而加入了 GNSS 观测信息对 IMU 机械编排进行更新时，不再出现误差发散情况，定位、定速和定姿误差均显著减小，波动幅度也更为稳定。

这次实验可以称得上是我进行的第一次组合导航实践，进行纯 IMU 解算时定位和定速误差会以极快的速度发散，发散速度与 IMU 性能强相关，这是 IMU 无法避免的一大缺陷。因此引入 GNSS 这一高精度观测值与 IMU 组合进行滤波处理，在 GNSS 信号存在时进行高精度测量值更新，修正 IMU 累计误差；在 GNSS 信号缺失时利用 IMU 数据进行机械编排以持续输出定位结果。组合导航就是利用了 GNSS 和 IMU 的互补优势，以适应更为广阔的应用场景。